Appendix B.
Anonymous Artifact Repository Material

B.1. Our ECPSS Scheme in the Client-Server Setting

An ECPSS scheme is parameterized by threshold t, committee size n, and total clients N. Our scheme also has an additional TTL parameter δ which denotes the duration after which a client that has announced its committee nomination to the server erases its encryption keys (§2.4). The client can later re-join the protocol by sampling fresh encryption keys (§5.2). This scheme uses the server-aided VSS scheme saVSS (Figure 13), and the transparency log ($\mathcal{F}^{\mathsf{tlog}}$, Figure 12) and PKI ($\mathcal{F}^{\mathsf{PKI}}$, Figure 11) ideal functionalities as building blocks. The CommitteeSelection procedure outputs a committee $\mathcal{C}$, where each committee member has an index (called seat in committee selection), and two public keys of interest: the corresponding client's encryption public key $\mathsf{pk}^{\mathsf{enc}}$ and signature public key $\mathsf{pk}^{\mathsf{sig}}$.

The remaining protocols Share, Reconstruct, and Handover build on committee selection, and are only called by the committee members in our key manager protocol (see Appendix A.3). As such, we'll assume that all of these clients have an index, an encryption public and a signature public key. If the protocol has a committee $\mathcal{C}$ as an input, then we assume that the caller of the protocol (i.e., the corresponding committee member) has verified $\mathcal{C}$ using the Get Committee function of committee selection before proceeding. To avoid confusion with other clients in the system, we denote a committee member with index k as $\mathcal{M}_k$ instead of $\mathcal{P}_k$.

We denote the j-th coefficient of the polynomial shared by $\mathcal{M}_i \in \mathcal{C}_{\mathsf{curr}}$ with $\mathsf{ccmt}_{\mathsf{curr},i,j}$. Given the coefficient commitments from t $\mathcal{C}_{\mathsf{curr}}$-members, anyone can interpolate (in the exponent) the commitment to $\mathcal{C}_{\mathsf{next}}$-members' shares. We denote the share commitment of $\mathcal{M}_i \in \mathcal{C}_{\mathsf{curr}}$ with $\mathsf{scmt}_{\mathsf{curr},i}$.

B.1.1. $\mathsf{ECPSS}_{t,n,N}.\mathsf{CommitteeSelection}(e)$**.** Let $\mathcal{P}$ denote the set of all clients. Recall from §5.2 that each party $\mathcal{P}_i$ has a unique index uid_i and three keypairs: a signature keypair $(\mathsf{pk}_i^{\mathsf{sig}}, \mathsf{sk}_i^{\mathsf{sig}})$ that is registered with $\mathcal{F}^{\mathsf{PKI}}$, a VRF keypair $(\mathsf{pk}_i^{\mathsf{vrf}}, \mathsf{sk}_i^{\mathsf{vrf}})$, an ephemeral encryption keypair $(\mathsf{pk}_i^{\mathsf{enc}}, \mathsf{sk}_i^{\mathsf{enc}})$. The set of public keys is referred to as ckeys_i. Whenever clients sample fresh encryption keys $(\mathsf{pk}_i^{\mathsf{enc}}, \mathsf{sk}_i^{\mathsf{enc}})$ after being on a committee (§5.2), we assume that they also send the server a well-formedness NIZK proof π_i^{enc} for these keys as the pVSS scheme we build on requires that the encryption keys be honestly generated. The goal of committee selection procedure is to output a committee $\mathcal{C}_e$ in epoch e such that only a small subset of the total clients are selected and any client can verifiably retrieve the public keys of the clients in this committee. We now describe the steps in committee selection in chronological order:

- **(Server) Setup Sortition**: Given a client list $\mathcal{P} = \{\mathcal{P}_i\}_i$ such that $\mathcal{F}^{\mathsf{PKI}}$ returns $\top$ on $\langle \mathsf{Verify}, \mathsf{uid}_i, \mathsf{pk}_i^{\mathsf{sig}} \rangle$ and π_i^{enc} is valid for $\mathsf{pk}_i^{\mathsf{enc}}$, construct a Merkle tree (hash function instantiated with random oracle) on the following dictionary $\{\mathsf{uid}_i : \mathsf{pk}_i^{\mathsf{vrf}}\}_{\mathcal{P}_i \in \mathcal{P}}$. Let rt denote the root of the resulting tree. Send rt to all clients in $\mathcal{P}$.
- **(Client) Check Sortition**: During epoch e[1], compute $\mathsf{seed} = H_{\mathsf{seed}}(\mathsf{rt}, e)$, where H_{seed} is a random oracle that has the same output space as the input space of the unbiasable VRF scheme. Check sortition result $\pi^{\mathsf{srt}} \leftarrow \mathsf{Sortition}(\mathsf{sk}^{\mathsf{vrf}}, \mathsf{seed}, n, N)$; sortition is configured to choose client with probability $\frac{n}{N}$. If $\pi^{\mathsf{srt}} = \perp$, return $\perp$. Otherwise, send π^{srt} to $\mathcal{S}$ and then start a local timer that will trigger a timeout after δ time. When the δ-timeout is triggered, delete $\mathsf{sk}_i^{\mathsf{enc}}$ if it was not already erased during handover.
- **(Server) Publish Committee**: For each sortition proof π_i^{srt} received from client $\mathcal{P}_i \in \mathcal{P}$ with keys ckeys_i, verify π_i^{srt} w.r.t. $(\mathsf{pk}_i^{\mathsf{vrf}}, \mathsf{rt}, e)$, and if successful, assign a unique index seat_i to the client (used to define Shamir shares), and add $\mathsf{mmbrDt}_i = \{\mathsf{uid}_i, \mathsf{seat}_i, \mathsf{ckeys}_i, \pi_i^{\mathsf{srt}}, \pi_i^{\mathsf{mt}}\}$ to the set mmbrsDt, where π_i^{mt} is the Merkle inclusion proof on committee members VRF keys w.r.t. rt. Return $\mathsf{cmtDt}_e = (\mathsf{rt}, \mathsf{mmbrsDt})$ and send $\langle \mathsf{Publish}, e \| \text{"cmt-data"}, \mathsf{cmtDt}_e \rangle$ to $\mathcal{F}^{\mathsf{tlog}}$.
- **(Client) Get Committee**: Get $\mathsf{cmtDt}_e = (\mathsf{rt}, \mathsf{mmbrsDt})$ from $\mathcal{S}$ and perform the following checks:
  - Send $\langle \mathsf{Verify}, e \| \text{"cmt-data"}, \mathsf{cmtDt}_e \rangle$ to $\mathcal{F}^{\mathsf{tlog}}$ and verify it responds with $\top$.
  - Verify that $|\mathsf{mmbrsDt} \geq t|$ and $\{\mathsf{uid}_i, \mathsf{seat}_i, \mathsf{ckeys}_i\}$ for each $\mathsf{mmbrDt}_i \in \mathsf{mmbrsDt}$ are distinct.
  - For each $\mathsf{mmbrDt}_i \in \mathsf{mmbrsDt}$, parse $\mathsf{mmbrDt}_i = \{\mathsf{uid}_i, \mathsf{seat}_i, \mathsf{ckeys}_i, \pi_i^{\mathsf{srt}}, \pi_i^{\mathsf{mt}}\}$, parse $\mathsf{ckeys}_i = (\mathsf{pk}_i^{\mathsf{sig}}, \mathsf{pk}_i^{\mathsf{vrf}}, \mathsf{pk}_i^{\mathsf{enc}})$, and perform the following checks:
    - Verify $\mathcal{F}^{\mathsf{PKI}}$ responds to $\langle \mathsf{Verify}, \mathsf{uid}_i, \mathsf{pk}_i^{\mathsf{sig}} \rangle$ with $\top$.
    - Verify π_i^{enc} is valid w.r.t. $\mathsf{pk}_i^{\mathsf{enc}}$.
    - Verify π_i^{mt} and check π_i^{srt} is valid w.r.t. $(\mathsf{pk}_i^{\mathsf{vrf}}, \mathsf{rt}, e)$.

  Return $\mathsf{cmtKeys}_e \leftarrow \{\mathsf{uid}_i, \mathsf{seat}_i, \mathsf{pk}_i^{\mathsf{enc}}, \mathsf{pk}_i^{\mathsf{sig}}\}_{\mathsf{mmbrDt}_i}$ if all checks pass, else return $\perp$.

1. Here e refers to the epoch maintained locally at the client which could be different from the system epoch maintained by the server. Server needs to ensure that the local and system epochs are (loosely) synchronized to ensure availability. Our security proof shows that a malicious server that does not synchronize the epochs can only affect availability and not privacy.

Server-Aided NIVSS $\mathsf{saVSS}_{n,t}$ in $\mathcal{F}^{\mathsf{tlog}}$-hybrid

n and t denote the number of recipients and the threshold of the secret-sharing scheme. sid denotes the share-id. $\mathcal{S}$ denotes the server, $\mathcal{D}_k$ the dealer, $\{\mathcal{R}_i\}_{i\in[n]}$ the recipients, and $\mathcal{F}^{\mathsf{tlog}}$ the transparency log (Figure 12). pVSS denotes the NIVSS scheme with public verifiability from [96] and aVSS denotes the NIVSS scheme with abort from [52]. Let $\mathsf{pp}^{\mathsf{pv}}$ and $\mathsf{pp}^{\mathsf{ab}}$ denote the corresponding public parameters chosen such that both schemes use the same dlog-hard group $\mathbb{G}$ with generator g and order q for commitments. Let $(\mathsf{pk}_i^{\mathsf{enc}}, \mathsf{sk}_i^{\mathsf{enc}})_{i\in[n]}$ denote the public key encryption (specified by $\mathsf{pp}^{\mathsf{pv}}$) keypairs of the recipients, and $(\mathsf{pk}_k^{\mathsf{sig}}, \mathsf{sk}_k^{\mathsf{sig}})$ denote the signature keypair of the dealer. Per the security requirement of pVSS [96], we assume that the encryption keys are well-formed and our ECPSS committee selection introduces checks to guarantee this (Appendix B.1.1).

$\mathsf{Deal}_{\mathcal{D}_k}(\mathsf{sid}, s, \{\mathsf{pk}_i^{\mathsf{enc}}\}_i, \mathsf{sk}_k^{\mathsf{sig}})$:

- $\forall j \in [t]$, sample $a_j \leftarrow \mathbb{F}_q$. Set $a_0 \leftarrow s$.
- $\forall i \in [n]$, set $s_i \leftarrow \sum_{j\in[t]} a_j \cdot (i+1)^j$.
- Commit to the coeffs $\{a_j\}_j$ to get $\mathsf{cmt} = \{g^{a_j}\}_j$.
- Encrypt the shares with recipients public keys and produce a proof of correct sharing: $(\mathsf{ct}, \pi^{\mathsf{pv}}) \leftarrow \mathsf{pVSS}_{n,t}.\mathsf{EncAndProve}(\mathsf{pp}^{\mathsf{pv}}, \mathsf{cmt}, \{s_i, \mathsf{pk}_i\}_i)$.
- Produce a NIVSS with abort proof w.r.t. cmt: $\pi^{\mathsf{ab}} \leftarrow \mathsf{aVSS}_{n,t}.\mathsf{Prove}(\mathsf{pp}^{\mathsf{ab}}, \{a_j\}_j, \mathsf{cmt})$.
- $\forall i \in [n]$, sign $(\mathsf{sid}, k, i, \mathsf{cmt}, \mathsf{ct}[i])$ using $\mathsf{sk}_k^{\mathsf{sig}}$ to get σ_i.
- Send $\mathsf{dlng} \leftarrow (\mathsf{cmt}, \mathsf{ct}, \{\sigma_i\}_i, \pi^{\mathsf{pv}}, \pi^{\mathsf{ab}})$ to $\mathcal{S}$.

$\mathsf{ProcessDealing}_{\mathcal{S}}(\mathsf{sid}, k, \mathsf{dlng}, \{\mathsf{pk}_i^{\mathsf{enc}}\}_i, \mathsf{pk}_k^{\mathsf{sig}})$:

- $\mathcal{S}$ parses $\mathsf{dlng} = (\mathsf{cmt}, \mathsf{ct}, \{\sigma_i\}_i, \pi^{\mathsf{pv}}, \pi^{\mathsf{ab}})$ and sends $(\mathsf{sid}, k, \perp)$ to $\{\mathcal{R}_i\}_i$ if any of the following check fails:
  - $\mathsf{pVSS}_{n,t}.\mathsf{Verify}(\mathsf{pp}^{\mathsf{pv}}, \mathsf{cmt}, \mathsf{ct}, \{\mathsf{pk}_i^{\mathsf{enc}}\}_i, \pi^{\mathsf{pv}})$
  - $\mathsf{aVSS}_{n,t}.\mathsf{Verify}(\mathsf{pp}^{\mathsf{ab}}, \mathsf{cmt}, \pi^{\mathsf{ab}})$
  - $\forall i$, verify σ_i w.r.t. $(\mathsf{sid}, k, i, \mathsf{cmt}, \mathsf{ct}[i])$ and $\mathsf{pk}_k^{\mathsf{sig}}$
- Else, send $\langle \mathsf{Publish}, \mathsf{sid}\|k, \mathsf{cmt}\rangle$ to $\mathcal{F}^{\mathsf{tlog}}$ and for $i \in [n]$, send $\mathsf{dlngLite}_i \leftarrow (\mathsf{cmt}, \mathsf{ct}[i], \sigma_i, \pi^{\mathsf{ab}})$ to $\mathcal{R}_i$.

$\mathsf{Receive}_{\mathcal{R}_i}(\mathsf{sid}, k, \mathsf{dlngLite}_i, \mathsf{sk}_i^{\mathsf{enc}}, \mathsf{pk}_k^{\mathsf{sig}}) \rightarrow \{s_i, \perp\}$:

- Parse $\mathsf{dlngLite}_i = (\mathsf{cmt}, \mathsf{ct}_i, \sigma, \pi^{\mathsf{ab}})$, decrypt ct_i with $\mathsf{sk}_i^{\mathsf{enc}}$ to get s_i, and perform the following checks:
  - $\mathsf{aVSS}_{n,t}.\mathsf{Verify}(\mathsf{pp}^{\mathsf{ab}}, \mathsf{cmt}, \pi^{\mathsf{ab}})$
  - Verify s_i w.r.t. cmt: $g^{s_i} = \prod_{j\in[t]} \mathsf{cmt}[j]^{(i+1)^j}$
  - Verify σ w.r.t. $(\mathsf{sid}, k, i, \mathsf{cmt}, \mathsf{ct})$ and $\mathsf{pk}_k^{\mathsf{sig}}$.
  - $\mathcal{F}^{\mathsf{tlog}}$ responds to $\langle \mathsf{Verify}, \mathsf{sid}\|k, \mathsf{cmt}\rangle$ with $\top$.
- If all checks pass, output s_i, else abort.

Figure 13. Our Server-Aided NIVSS Scheme

B.1.2. $\mathsf{ECPSS}_{t,n,N}.\mathsf{Share}(s, \mathcal{C})$**.** Let some committee member $\mathcal{M}_k$ be the caller of this API in epoch e. Let $\mathsf{PK} = \{\mathsf{pk}_i^{\mathsf{enc}}\}_{\mathcal{M}_i\in\mathcal{C}}$ denote the encryption public keys of the committee $\mathcal{C}$-members and $\mathsf{pk}_k^{\mathsf{sig}}$ denote the signature public key of $\mathcal{M}_k$. The sharing proceeds as follows:

- If δ-timeout has already been triggered for $\mathcal{M}_k$, it aborts. Else, $\mathcal{M}_k$ invokes $\mathsf{saVSS}.\mathsf{Deal}(e, s, \mathsf{PK}, \mathsf{sk}_k^{\mathsf{sig}})$ to generate a dealing dlng_k that shares secret s to the committee $\mathcal{C}$ while setting session-id $\mathsf{sid} = e$, which is then sent to the server $\mathcal{S}$ for processing.
- The server calls $\mathsf{saVSS}.\mathsf{ProcessDealing}(e, k, \mathsf{dlng}_k, \mathsf{PK}, \mathsf{pk}_k^{\mathsf{sig}})$ to get lite-dealings $\{\mathsf{dlngLite}_{k,i}\}_{\mathcal{M}_i\in\mathcal{C}}$ for each committee $\mathcal{C}$-member. The server then sends $\mathsf{dlngLite}_{k,i}$ to $\mathcal{M}_i \in \mathcal{C}$ when they come online.

B.1.3. $\mathsf{ECPSS}_{t,n,N}.\mathsf{Reconstruct}(\mathcal{C})$**.** Let $\mathcal{M}_i$ be a committee member that is supposed to receive a secret from committee $\mathcal{C}$ that was shared in epoch e. Let $\{\mathsf{pk}_k^{\mathsf{sig}}\}_{\mathcal{M}_k\in\mathcal{C}}$ denote the signature public keys of the the committee $\mathcal{C}$-members. The reconstruction proceeds as follows:

- $\mathcal{M}_i$ waits to receive t lite-dealings $\{\mathsf{dlngLite}_{k,i}\}_{\mathcal{M}_k\in\mathcal{C}'}$ from the server where $\mathcal{C}' \subset \mathcal{C}$ and $|\mathcal{C}'| = t$. If the δ-timeout is triggered before it receives the lite-dealings, $\mathcal{M}_i$ aborts.
- $\forall \mathcal{M}_k \in \mathcal{C}'$, $\mathcal{M}_i$ invokes $s_k \leftarrow \mathsf{saVSS}.\mathsf{Receive}(e, k, \mathsf{dlngLite}_{k,i}, \mathsf{sk}_i^{\mathsf{enc}}, \mathsf{pk}_k^{\mathsf{sig}})$. If any Receive call aborts, $\mathcal{M}_k$ aborts.
- Otherwise, given $\{i, s_i\}_{\mathcal{M}_i\in\mathcal{C}'}$, $\mathcal{M}_k$ interpolates the secret s (using indices i as the evaluation points during Shamir reconstruction), and outputs s.

B.1.4. $\mathsf{ECPSS}_{t,n,N}.\mathsf{Handover}(\mathcal{C}_{\mathsf{prev}}, \mathcal{C}_{\mathsf{curr}}, \mathcal{C}_{\mathsf{next}})$**.** During epoch e, server $\mathcal{S}$ initiates the handover, where committee $\mathcal{C}_{\mathsf{curr}}(\mathsf{curr} = e-2)$ receives the secret (msk of the IBE scheme in Appendix A.3) from $\mathcal{C}_{\mathsf{prev}}(\mathsf{prev} = e-3)$, and re-shares it to committee $\mathcal{C}_{\mathsf{next}}(\mathsf{next} = e-1)$. In addition to performing the handover, $\mathcal{C}_{\mathsf{curr}}$-members also verify that the handover generated by $\mathcal{C}_{\mathsf{prev}}$ is consistent with the handover received by $\mathcal{C}_{\mathsf{prev}}$. To aid this verification, the server $\mathcal{S}$ publishes the share commitments for each committee to the transparency log, and the receiving committee members verify that each sender committee member created a dealing consistent with its share commitment. Moreover, the receiving committee members verify the share commitments the server posted for them and only perform the handover if these commitments are consistent with the handover from the sender committee. In more detail, the handover proceeds as follows:

- Each (online) party $\mathcal{M}_i \in \mathcal{C}_{\mathsf{curr}}$ performs the following operations (aborting if $\mathsf{Share}, \mathsf{Reconstruct}$ abort or any of the checks fail):
  - Invoke $\mathsf{msk}_i \leftarrow \mathsf{ECPSS}_{t,n,N}.\mathsf{Reconstruct}(\mathcal{C}_{\mathsf{prev}})$, and receive $\mathsf{scmt}_{\mathsf{prev}}$ and $\mathsf{scmt}_{\mathsf{curr}}$ in addition to the handover from $\mathcal{S}$.
  - Verify that $\mathcal{F}^{\mathsf{tlog}}$ responds with $\top$ on queries $\langle \mathsf{Verify}, \mathsf{prev} \| \text{“scmt”}, \mathsf{scmt}_{\mathsf{prev}} \rangle$ and $\langle \mathsf{Verify}, \mathsf{curr} \| \text{“scmt”}, \mathsf{scmt}_{\mathsf{curr}} \rangle$.
  - $\forall \mathcal{M}_k \in \mathcal{C}_{\mathsf{prev}}$, verify that $\mathsf{ccmt}_{\mathsf{prev},k,0} = \mathsf{scmt}_{\mathsf{prev},k}$.
  - Interpolate $\mathsf{scmt}'_{\mathsf{curr}}$ from $\{\mathsf{ccmt}_{\mathsf{prev},k,j}\}_{\mathcal{M}_k \in \mathcal{C}_{\mathsf{prev}}, j \in [t]}$ and verify that $\mathsf{scmt}_{\mathsf{curr}} = \mathsf{scmt}'_{\mathsf{curr}}$.
  - Invoke $\mathsf{ECPSS}_{t,n,N}.\mathsf{Share}(\mathsf{msk}_i, \mathcal{C}_{\mathsf{next}})$ and erase the share msk_i. Also erase $\mathsf{sk}_i^{\mathsf{enc}}$ if it was not already deleted due to the local δ-timeout.
- As part of the Share invocations by members of $\mathcal{C}_{\mathsf{curr}}$, server $\mathcal{S}$ receives the dealings generated by $\mathcal{C}_{\mathsf{curr}}$. In addition to the saVSS validity checks performed independently on these dealings in Share, $\mathcal{S}$ also checks that these dealings are consistent with the share commitments for those committee members. It then interpolates $\mathsf{scmt}_{\mathsf{next}}$ from the current handover and sends $\langle \mathsf{Publish}, \mathsf{next} \| \text{“scmt”}, \mathsf{scmt}_{\mathsf{next}} \rangle$ to $\mathcal{F}^{\mathsf{tlog}}$ to publish the share-commitments for $\mathcal{C}_{\mathsf{next}}$. Finally, $\mathcal{S}$ forwards only the lite-dealings corresponding to dealings that are both consistent and valid to $\mathcal{C}_{\mathsf{next}}$ (t valid and consistent dealings are sufficient).

B.2. anon-IND-ID-CCA property of BF-IBE

In this section, we define the anon-IND-ID-CCA property satisfied by the BF-IBE scheme [33]. This property was proven by Paterson and Srinivasan [123] (henceforth referred to as [PS08]) for a slight variant of BF-IBE in the multiple trusted authority (TA) setting. Specifically, they prove that this BF-IBE variant satisfies the m-RA-IND-ID-CCA property defined in [PS08]. This property is identical to the anon-IND-ID-CCA property we define below when #TA is set to 1. Throughout the paper and in our implementation, BF-IBE denotes the [PS08] variant with #TA=1.

Let $\mathbf{G}_{\mathsf{IBE}}^{\mathsf{anon\text{-}IND\text{-}ID\text{-}CCA}-b}$ denote the anon-IND-ID-CCA game w.r.t. an IBE scheme, and let $W_b(\mathcal{A})$ denote the event that the game $\mathbf{G}_{\mathsf{IBE}}^{\mathsf{anon\text{-}IND\text{-}ID\text{-}CCA}-b}(\mathcal{A})$ outputs 1. Then we call IBE anon-IND-ID-CCA-secure if for all non-uniform PPT adversaries $\mathcal{A}$, $\Pr[W_0(\mathcal{A})] - \Pr[W_1(\mathcal{A})]| \leq \mathrm{negl}(\lambda)$.

$\mathbf{G}_{\mathsf{IBE}}^{\mathsf{anon\text{-}IND\text{-}ID\text{-}CCA}-b}(\mathcal{A})$	$\textsc{Ext}(\mathsf{id})$
$(\mathsf{msk}, \mathsf{pk}) \leftarrow \mathsf{IBE.KeyGen}(1^\lambda)$	Add id to E
$(\mathsf{id}_0, \mathsf{id}_1, m_0, m_1) \leftarrow \mathcal{A}^{\textsc{Ext},\textsc{Dec}}(\mathsf{pk})$	**return** $\mathsf{IBE.Extract}(\mathsf{msk}, \mathsf{id})$
$\mathsf{ct}^* \leftarrow \mathsf{IBE.Encrypt}(\mathsf{pk}, \mathsf{id}_b, m_b)$	$\textsc{Dec}(\mathsf{id}, \mathsf{ct})$
$b' \leftarrow \mathcal{A}^{\textsc{Ext},\textsc{Dec}}(\mathsf{ct}^*)$	Add ct to D
return $b' \wedge \mathsf{id}_0, \mathsf{id}_1 \notin E \wedge \mathsf{ct}^* \notin D$	$d_{\mathsf{id}} \leftarrow \mathsf{IBE.Extract}(\mathsf{msk}, \mathsf{id})$
	return $\mathsf{IBE.Decrypt}(d_{\mathsf{id}}, \mathsf{ct})$

B.3. Security of the Secret Recovery Scheme

In this section, we prove privacy (Appendix B.3.1) and correctness (Appendix B.3.2) of our secret recovery scheme.

B.3.1. Privacy of the secret recovery scheme. The secrets in our system are secured using passwords which do not follow a uniform distribution over the password space $\mathbb{P}$. Thus, inspired by prior works on password-based primitives [19], [24], [26], we define our privacy game $\mathbf{G}_{\mathcal{PW},L}^{\mathsf{SR,priv}}$ w.r.t. a password distribution space $\mathcal{PW}$ that outputs a password vector, containing a password from $\mathbb{P}$ for each user in the system. Our privacy game (Figure 14) allows the adversary to corrupt the server and an arbitrary subset of clients except one honest client $\mathcal{P}_{\mathsf{uid}^*}$. The adversary can also ask $\mathcal{P}_{\mathsf{uid}^*}$ and other honest clients to perform arbitrary sequences of backups and recoveries. We define a scheme to be private if with all these capabilities, the best an adversary can do to win the game is guess the password within L attempts given knowledge of distribution $\mathcal{PW}$. Now we formally state our definition.

Privacy Game $\mathbf{G}^{\mathsf{SR,priv}}_{\mathcal{PW},L}$ for Key Recovery

This game is parameterized by a password distribution $\mathcal{PW}$ and password guess limit L, and defined w.r.t. a key recovery scheme $\mathsf{SR}_L = (\mathsf{KeyGen}, \mathsf{Backup}, \mathsf{Recover})$. During the game, the challenger and the adversary $\mathcal{A}$ play the role of the honest and corrupted entities, resp., in SR_L protocols.

- **Setup**. Challenger samples a vector of passwords $\mathsf{PW} \leftarrow_{\$} \mathcal{PW}$ for each user of the system, and the challenger and $\mathcal{A}$ invoke $\mathsf{SR}_L.\mathsf{KeyGen}()$.
- **Queries**. At any point during the game (before or after the challenge), $\mathcal{A}$ can make the following oracle queries:
  - $\mathsf{Backup}(\mathsf{uid}, \mathsf{msg})$: challenger plays the role of honest client $\mathcal{P}_{\mathsf{uid}}$ and invokes $\mathsf{SR}_L.\mathsf{Backup}(\mathsf{uid}, \mathsf{PW}[\mathsf{uid}], \mathsf{msg})$.
  - $\mathsf{Recover}(\mathsf{uid})$: challenger plays the role of honest client $\mathcal{P}_{\mathsf{uid}}$ and invokes $\mathsf{SR}_L.\mathsf{Recover}(\mathsf{uid}, \mathsf{PW}[\mathsf{uid}])$.
  - $\mathsf{Corrupt}(\mathsf{uid})$: $\mathcal{A}$ receives $\mathsf{PW}[\mathsf{uid}]$ from the challenger and it now controls client $\mathcal{P}_{\mathsf{uid}}$.
- **Challenge**. The adversary sends two messages $\mathsf{msg}_0, \mathsf{msg}_1$ s.t. $|\mathsf{msg}_0| = |\mathsf{msg}_1|$ and a user identity uid^* to the challenger. The challenger samples a random bit $b \leftarrow_{\$} \{0, 1\}$ and calls $\mathsf{Backup}(\mathsf{uid}^*, \mathsf{msg}_b)$.
- **Output**. Finally, the adversary outputs a bit b' and wins the game if $b = b'$ and no $\mathsf{Corrupt}$ queries were made for uid^*.

Figure 14. Privacy of the key recovery scheme.

Password Guessing Game $\mathbf{G}^{\mathsf{PG}}_{\mathcal{PW},L}$

This game is parameterized by a password distribution $\mathcal{PW}$ and password guess limit L, where $\mathcal{PW}$ outputs a vector containing a password for each user.

- **Setup**. Challenger samples a vector of passwords for each user of the system: $\mathsf{PW} \leftarrow_{\$} \mathcal{PW}$.
- **Queries**. At any point after the setup (before or after the challenge), $\mathcal{A}$ can make the following queries:
  - $\mathsf{Guess}(\mathsf{uid}, \mathsf{pwd})$: challenger sends $\top$ to $\mathcal{A}$ if $\mathsf{pwd} = \mathsf{PW}[\mathsf{uid}]$ and $\perp$ otherwise.
  - $\mathsf{Reveal}(\mathsf{uid})$: $\mathcal{A}$ receives $\mathsf{PW}[\mathsf{uid}]$ from the challenger.
- **Challenge**. $\mathcal{A}$ sends uid^* to the challenger.
- **Output**. Finally, $\mathcal{A}$ outputs the password guess pwd^*. If $\mathsf{pwd}^* = \mathsf{PW}[\mathsf{uid}^*]$, and $\mathcal{A}$ made $\leq L$ Guess queries and no Reveal queries for uid^*, $\mathcal{A}$ wins the game.

Figure 15. Password Guessing Game

Definition 1. Let $\mathsf{SR}_L = (\mathsf{KeyGen}, \mathsf{Backup}, \mathsf{Recover})$ parameterized by security parameter λ and guess limit L be a secret recovery scheme and let $\mathbf{G}^{\mathsf{SR,priv}}_{\mathcal{PW},L}$ be the secret recovery privacy game (Figure 14) for SR_L w.r.t. a password distribution $\mathcal{PW}$. We define the advantage of an adversary $\mathcal{A}$ playing this game as follows: $\mathbf{Adv}^{\mathsf{SR,priv}}_{\mathcal{PW},L}(\mathcal{A}) := |\ \Pr[\mathcal{A} \text{ wins } \mathbf{G}^{\mathsf{SR,priv}}_{\mathcal{PW},L}] - 1/2\ |$.

Let $\mathbf{Adv}^{\mathsf{PG}}_{\mathcal{PW},L}(\mathcal{B}^{\mathsf{PG}})$ denote the winning probability of an adversary $\mathcal{B}^{\mathsf{PG}}$ in the password-guessing game $\mathbf{G}^{\mathsf{PG}}_{\mathcal{PW},L}$ (Figure 15). Then SR_L is a *private* secret recovery scheme if $\forall$ PPT adversaries $\mathcal{A}$, $\exists\ \mathcal{B}^{\mathsf{PG}}$ s.t. $\mathbf{Adv}^{\mathsf{SR,priv}}_{\mathcal{PW},L}(\mathcal{A}) \leq \mathbf{Adv}^{\mathsf{PG}}_{\mathcal{PW},L}(\mathcal{B}^{\mathsf{PG}}) + \mathrm{negl}(\lambda)$.

Theorem 2. *The secret recovery scheme (Figure 3) is private as per Definition 1 for any PPT adversary that corrupts the server $\mathcal{S}$ assuming the $\mathcal{F}^{\mathsf{KM}}$ functionality from Figure 2.*

Proof. Consider the following sequence of hybrids, where the difference in winning probability between consecutive hybrids is negligible in λ unless stated otherwise:

- H_0 **(The real game $\mathbf{G}^{\mathsf{SR,priv}}_{\mathcal{PW},L}$)**: this is the same as the challenger and the adversary $\mathcal{A}$ playing the privacy game, where the invocations to the secret recovery scheme are performed as specified by the protocol. Since the challenger plays the role of honest parties, it knows msk of the IBE scheme (on behalf of $\mathcal{F}^{\mathsf{KM}}$) and the trapdoor td w.r.t. the honestly generated CRS for the zkSNARK [119].
- H_1 **(Zero-knowledge of zkSNARK [119])**: the challenger replaces honest client's zkSNARK proofs with simulated proofs using the trapdoor td.
- H_2 **(The blind is a one-time pad)**: the challenger replaces rqst for honest clients with a random $\mathbb{G}_2$ element. Since the challenger already replaced the zkSNARK proof with a simulated proof, it can generate a proof for a random rqst without knowing the witness. This is identical to the previous hybrid from the adversary's perspective because the blind r perfectly hides the recovery request ($\mathbb{G}_2$ is of prime-order). At this point, the honest client's recovery requests do not reveal any additional information besides the uid.

- H_3 **(Invariant Soundness of authenticated dictionary [140])**: the challenger aborts if $\mathcal{F}^{\mathsf{KM}}$ responds to $> L$ Recover requests for any uid.
- H_4 **(Weak Simulation-extractability of zkSNARK [119])**: the challenger aborts if it can not extract (pwd, r) from the recovery request made by the adversary.
- H_5 **(anon-IND-ID-CCA prop. of the IBE scheme; Appendix B.2)**:
  - The challenger aborts if the adversary makes a recovery request with inputs $\mathsf{id}^* = \mathsf{uid}^* \| \mathsf{PW}[\mathsf{uid}^*]$. It can detect the abort condition because it knows $(\mathsf{uid}, \mathsf{pwd})$ for all of adversary's recovery requests (from H_4).
  - The challenger samples a random identity id' and AE key k', and replaces the IBE ciphertext $\mathsf{ct}_{\mathsf{ibe}}$ in the challenge backup with $\mathsf{ct}'_{\mathsf{ibe}} \leftarrow \mathsf{IBE.Encrypt}(\mathsf{pk}, \mathsf{id}', k')$.

  Note that this hybrid contains no information about $\mathsf{PW}[\mathsf{uid}^*]$ and the AE key k used to encrypt msg_b. In Lemma 3, we prove that (i) adversary's probability of guessing the right password in H_4 and H_5 is negligibly different, (ii) the adversary can't distinguish the hybrids if it does not guess the password, and (iii) adversary's probability of guessing the right password in H_5 is $\mathbf{Adv}^{\mathsf{PG}}_{\mathcal{PW},L}$. All three combined show that adversary's advantage in distinguishing H_4 and H_5 is $\leq \mathbf{Adv}^{\mathsf{PG}}_{\mathcal{PW},L} + \mathrm{negl}(\lambda)$.
- H_6 **(IND-CCA2 prop. of Authenticated Encryption [23])**: the challenger replaces msg_b with a random message msg' in the AE ciphertext $\mathsf{ct}_{\mathsf{ae}}$ of the challenge backup.

Overall, this shows that the adversary's probability of distinguishing H_0 and H_6 is $\leq \mathbf{Adv}^{\mathsf{PG}}_{\mathcal{PW},L} + \mathrm{negl}(\lambda)$. Since the final hybrid contains no information about b, the adversary can only guess the bit and this completes the proof. □

Lemma 3. *Let W_4 and W_5 denote the adversary $\mathcal{A}$'s winning probability in Hybrid H_4 and H_5 from the proof of Theorem 2, $\forall \mathcal{A}$, $\exists\ \mathcal{B}^{\mathsf{PG}}$ s.t. $|\Pr[W_4] - \Pr[W_5]| \leq \mathbf{Adv}^{\mathsf{PG}}_{\mathcal{PW},L}(\mathcal{B}^{\mathsf{PG}}) + \mathrm{negl}(\lambda)$.*

Proof. Let E_i denote the event that the adversary makes a recovery request for id^* in hybrid H_i. Then we can write:

$$\Pr[W_4] = \Pr[E_4] \cdot \Pr[W_4|E_4] + \Pr[\neg E_4] \cdot \Pr[W_4|\neg E_4]$$

Similarly, we can write:

$$\begin{aligned}\Pr[W_5] &= \Pr[E_5] \cdot \Pr[W_5|E_5] + \Pr[\neg E_5] \cdot \Pr[W_5|\neg E_5] \\ &= \Pr[\neg E_5] \cdot \Pr[W_5|\neg E_5] \qquad (\Pr[W_5|E_5] = 0)\end{aligned}$$

Now, we argue the following: (i) $|\Pr[E_4] - \Pr[E_5]| \leq \mathrm{negl}(\lambda)$, (ii) $|\Pr[W_4|\neg E_4] - \Pr[W_5|\neg E_5]| \leq \mathrm{negl}(\lambda)$, and (iii) there exists $\mathcal{B}^{\mathsf{PG}}$ such that $\Pr[E_5] = \mathbf{Adv}^{\mathsf{PG}}_{\mathcal{PW},L}(\mathcal{A})$, which implies:

$$\begin{aligned}&|\Pr[W_4] - \Pr[W_5]| \\ &\quad\leq |\Pr[E_4] \cdot \Pr[W_4|E_4]| \\ &\qquad + \ |\Pr[\neg E_4] \cdot \Pr[W_4|\neg E_4] - \Pr[\neg E_5] \cdot \Pr[W_5|\neg E_5]| \\ &\quad\leq |\Pr[E_4]| + |\Pr[\neg E_4] \cdot (\Pr[W_4|\neg E_4] - \Pr[W_5|\neg E_5])| \\ &\qquad + \ |(\Pr[\neg E_4] - \Pr[\neg E_5]) \cdot \Pr[W_5|\neg E_5]| \\ &\quad\leq |\Pr[E_5] + (\Pr[E_4] - \Pr[E_5])| + |\Pr[\neg E_4] \cdot \mathrm{negl}(\lambda)| \\ &\qquad + \ |(\Pr[E_5] - \Pr[E_4]) \cdot \Pr[W_5|\neg E_5]| \\ &\quad\leq \ \Pr[E_5] + \mathrm{negl}(\lambda) \leq \mathbf{Adv}^{\mathsf{PG}}_{\mathcal{PW},L}(\mathcal{B}^{\mathsf{PG}}) + \mathrm{negl}(\lambda)\end{aligned}$$

$\underline{|\Pr[E_4] - \Pr[E_5]| \leq \mathrm{negl}(\lambda)}$: To prove this, we consider a modified hybrid H'_i which is the same as H_i except the adversary $\mathcal{A}$ wins if and only if event E_i happens. We show that if $\mathcal{A}$ has a non-negligible advantage ϵ in winning in H'_4 compared to H'_5, then we can construct an adversary $\mathcal{B}$ that uses $\mathcal{A}$ in a black-box way to win the anon-IND-ID-CCA game of the IBE scheme, leading to a contradiction. $\mathcal{B}$ works as follows: it behaves the same way as the H'_4/H'_5 challenger except it doesn't know msk of the IBE scheme. Nevertheless, it can perform the role of $\mathcal{F}^{\mathsf{KM}}$ by querying the (outside) anon-IND-ID-CCA challenger through extraction and decryption queries. Even though the recovery requests to $\mathcal{F}^{\mathsf{KM}}$ are blinded, the challenger knows the blind r and $\mathsf{id} = \mathsf{pwd} \| \mathsf{uid}$ for each recovery request in both hybrid H'_4/H'_5. Thus, it can simply ask for extraction on id from the outside challenger and then blind the response with r before forwarding it to $\mathcal{A}$. When $\mathcal{A}$ chooses a challenge uid^* and messages msg_0 and msg_1, $\mathcal{B}$ does the following: it samples a bit b, a random identity id', and random AE keys k, k'. It then encrypts msg_b with key k to get $\mathsf{ct}^*_{\mathsf{ae}}$ and chooses the challenge identities and messages for the outside challenger as $\mathsf{id}_0 = \mathsf{id}^*, \mathsf{id}_1 = \mathsf{id}', m_0 = k, m_1 = k'$. The outside challenger chooses a bit b', and sends the challenge ciphertext $\mathsf{ct}^*_{\mathsf{ibe}}$ to $\mathcal{B}$. In response to $\mathcal{A}$'s challenge, $\mathcal{B}$ sets $\mathsf{ct}_{\mathsf{uid}^*} = (\mathsf{ct}^*_{\mathsf{ibe}}, \mathsf{ct}^*_{\mathsf{ae}})$ as the ciphertext during challenge backup. Note that from the perspective of $\mathcal{A}$, if the outside challenger picked $b' = 0$, then we're exactly in H'_4 and if $b' = 1$, then we're exactly in H'_5. If event E happens, i.e., $\mathcal{A}$ makes a recovery request for id^*, then $\mathcal{B}$ outputs

Correctness Game $\mathbf{G}_L^{\mathsf{SR,corr}}$ for Secret Recovery

This game is parameterized by password guess limit L and a secret recovery scheme $\mathsf{SR}_L = (\mathsf{KeyGen}, \mathsf{Backup}, \mathsf{Recover})$. The challenger plays the role of all parties in SR_L protocols.

- **Setup**. same as protocol; challenger sends pk to $\mathcal{A}$.
- **Challenge**. The adversary sends msg^*, $\mathsf{uid}^* \in \mathbb{U}$, $\mathsf{pwd}^* \in \mathbb{P}$, and L password guesses $\{\mathsf{pwd}_i\}_{i\in[L]}$ to the challenger. On behalf of $\mathcal{P}_{\mathsf{uid}^*}$, the challenger calls $\mathsf{SR}_L.\mathsf{Backup}(\mathsf{uid}^*, \mathsf{pwd}^*, \mathsf{msg}^*)$, and then for each password guess pwd_i, the challenger calls $\mathsf{msg}_i \leftarrow \mathsf{SR}_L.\mathsf{Recover}(\mathsf{uid}^*, \mathsf{pwd}_i)$.
- **Output**. The adversary wins the game if $\exists\ i \in [L]$ s.t. $\mathsf{pwd}_i = \mathsf{pwd}^*$ and $\mathsf{msg}_i \neq \mathsf{msg}^*$.

Figure 16. Correctness of the secret recovery scheme.

0, else it outputs 1. If the winning probability in H_4' and H_5' differs by non-negligible probability ϵ, then it is immediate that the probability that $\mathcal{B}$ outputs 1 in case $b' = 0$ and $b' = 1$ also differs by non-negligible probability.

$|\Pr[W_4|\neg E_4] - \Pr[W_5|\neg E_5]| \leq \mathrm{negl}(\lambda)$: This proof follows through a similar reduction to the previous one. We can construct $\mathcal{B}$ that wins the anon-IND-ID-CCA game using an adversary $\mathcal{A}$ that has a non-negligible advantage in winning in H_4 over H_5 (conditioned on not making a recovery request for id^*). $\mathcal{B}$ works the same as the previous reduction, except $\mathcal{B}$ outputs 0 if $\mathcal{A}$ guesses b correctly, else it outputs 1. Since the winning condition requires not querying $\mathsf{id}_0 = \mathsf{id}^*$ and $\mathcal{A}$ has negligible probability of guessing $\mathsf{id}_1 = \mathsf{id}'$, $\mathcal{B}$ never has to query outside challenger on $\mathsf{id}_0, \mathsf{id}_1$. Finally, if the winning probability in H_4 and H_5 differs by non-negligible probability ϵ, then it is immediate that the probability that $\mathcal{B}$ outputs 1 in case $b' = 0$ and $b' = 1$ also differs by non-negligible probability.

$\Pr[E_5] = \mathbf{Adv}^{\mathsf{PG}}_{\mathcal{PW},L}(\mathcal{B}^{\mathsf{PG}})$: Since H_5 contains no information about the password $\mathsf{PW}[\mathsf{uid}^*]$, it is easy to show a reduction to the password guessing game. In particular, we can show that if $\mathcal{A}$ makes a recover request for id^* with ϵ probability, then we can construct adversary $\mathcal{B}^{\mathsf{PG}}$ that has the same probability to win the password guessing game. $\mathcal{B}^{\mathsf{PG}}$ works as follows: it faithfully performs the role of challenger in H_5, except it doesn't sample PW during Setup, and instead, relies on queries of the password guessing game to answer the queries of the privacy game. The $\mathsf{Corrupt}$ queries correspond directly to the Reveal query in the password guessing game, and the Backup and $\mathsf{Recover}$ queries can also be answered for $\mathsf{uid} \neq \mathsf{uid}^*$ through the Reveal query. To answer Backup and $\mathsf{Recover}$ queries on uid^* in H_5, note that we don't need to know $\mathsf{PW}[\mathsf{uid}^*]$. When $\mathcal{A}$ performs recovery request for $(\mathsf{pwd}, \mathsf{uid}^*)$, we use the Guess query to see if it is the correct password. If Guess returns $\top$, we know that event E_5 occurred, and we return the corresponding password as the output of the password guessing game. Since $\Pr[E_5] = \epsilon$, $\mathcal{B}^{\mathsf{PG}}$ should also win the game by probability ϵ, and thus we arrive at a contradiction. □

B.3.2. Correctness of the secret recovery scheme. We define the correctness game $\mathbf{G}_L^{\mathsf{SR,corr}}$ for a secret recovery scheme SR in Figure 16. The game models that if the server is honest (meaning we have availability), for all $\mathsf{msg}^* \in \{0,1\}^*$, $\mathsf{uid}^* \in \mathbb{U}$, $\mathsf{pwd}^* \in \mathbb{P}$, honest user $\mathcal{P}_{\mathsf{uid}^*}$ will always be able to retrieve its secret message msg^* provided that it guesses the right password within L attempts. Here, like prior work on password-based secret recovery, we assume that an honest server will employ alternative authentication mechanisms to prevent the attacker from exhausting the guess limit for an honest user (see "Authentication Server" in [55]).

Definition 4. Let $\mathsf{SR}_L = (\mathsf{KeyGen}, \mathsf{Backup}, \mathsf{Recover})$ be a secret recovery scheme parameterized by security parameter λ and guess limit L. Let $\mathbf{G}_L^{\mathsf{SR,corr}}$ be the secret key recovery correctness game (Figure 16) for SR_L. We define the advantage of an adversary $\mathcal{A}$ playing this game as follows: $\mathbf{Adv}_L^{\mathsf{SR,corr}}(\mathcal{A}) := \Pr[\mathcal{A} \text{ wins } \mathbf{G}_L^{\mathsf{SR,corr}}]$

Then SR_L is a *correct* key recovery scheme if for all PPT adversaries $\mathcal{A}$, $\mathbf{Adv}_L^{\mathsf{SR,corr}}(\mathcal{A}) = 0$.

Theorem 5. *The key recovery scheme (defined in Figure 3) is correct as per Definition 4 for any PPT adversary assuming the $\mathcal{F}^{\mathsf{KM}}$ functionality from Figure 2 and the server $\mathcal{S}$ is honest.*

The proof follows straightforwardly from the correctness of the IBE scheme and the AE scheme, and the completeness of zkSNARK.

B.4. Security of the KM protocol

In this section, we prove that the KM protocol described in Appendix A.3 securely realizes the $\mathcal{F}^{\mathsf{KM}}$ functionality described in Figure 2.

B.4.1. Threat Model. We begin by defining the capabilities of the environment $\mathcal{Z}$ and the adversary $\mathcal{A}$: $\mathcal{Z}$ and $\mathcal{A}$ interact throughout the protocol. $\mathcal{Z}$ is responsible for choosing honest party inputs, when they make protocol or ideal functionality API invocations, and it gets the honest party outputs. $\mathcal{A}$ can corrupt at most κ fraction of the total clients. It is mobile and the corruption it makes take effect after δ time. $\mathcal{A}$ can arbitrarily schedule and drop messages with the constraint that for all e, the messages exchanged among the following honest parties during epochs e through $e+2$ are delivered within δ time from the start of epoch e: the honest server, the ideal functionalities, and $1-\phi$ fraction of honest clients (see §2.4). We model this by assuming that all parties (including ideal functionalities) are connected through a star topology, at the center of which is a router that $\mathcal{A}$ controls.

We assume a constraint on $\mathcal{A}$'s ability to control the network for simplicity. If this assumption is violated, we claim that this only affects availability and not privacy (similarly to how a corrupted server only affects availability). To see why, note that our modeling allows $\mathcal{A}$ to corrupt the server $\mathcal{S}$, which in the real protocol serves the same role as the router, and as such, can arbitrarily control message delivery and drop messages without any constraints. Nevertheless, we introduce the router in our model to capture the security of our protocol in case the server is not corrupted.

B.4.2. Definition. Before we state the formal definition, we first define the ideal and real world experiments. Both experiments are parameterized by the security parameter λ, the password guess-limit L, and the auxiliary input to the environment $z \in \{0,1\}^{\mathrm{poly}(\lambda)}$.

The ideal experiment. In the ideal world, the honest parties interact with $\mathcal{F}^{\mathsf{KM}}$ and the adversary $\mathcal{A}$ interacts with the environment $\mathcal{Z}$ and the ideal-world adversary (or simulator) Sim which simulates the honest parties for $\mathcal{A}$. $\mathcal{A}$ controls the corrupted parties. $\mathcal{Z}$ specifies the inputs of the honest parties. If the server $\mathcal{S}$ is honest, $\mathcal{Z}$ adaptively specifies when KeyGen and the subsequent $\mathsf{Recover}$ APIs are called and the inputs to the $\mathsf{Recover}$ invocation. The honest server adds any $\mathsf{Recover}$ calls to a queue and executes them in order after calling KeyGen. Although the real protocol processes a batch of requests at a time, we assume a single request is processed at a time in Figure 2 and in this proof for simplicity. $\mathcal{Z}$ also specifies which honest parties $\mathcal{P}$ will call the $\mathsf{PublicKey}$ API and it can arbitrarily interleave these requests. At the end of the experiment ($\mathcal{Z}$ decides when), $\mathcal{Z}$ outputs a bit, which we denote by $\mathrm{IDEAL}_{\mathcal{F}^{\mathsf{KM}},\mathsf{Sim},\mathcal{Z}}(\lambda, L, z)$.

The real experiment. In the real world, the (honest) parties send messages to each other and the ideal functionalities $\mathcal{F}^{\mathsf{tlog}}, \mathcal{F}^{\mathsf{PKI}}$ according to a specified key manager protocol KM. The adversary $\mathcal{A}$ interacts with the honest parties and $\mathcal{F}^{\mathsf{tlog}}, \mathcal{F}^{\mathsf{PKI}}$ in this world in addition to the environment $\mathcal{Z}$. Identical to the ideal experiment, $\mathcal{A}$ controls the corrupted parties, and $\mathcal{Z}$ specifies the inputs of honest parties with the same constraints and looks at their outputs. At the end of the experiment, $\mathcal{Z}$ outputs a bit $\mathrm{REAL}^{\mathcal{F}^{\mathsf{tlog}},\mathcal{F}^{\mathsf{PKI}}}_{\mathsf{KM},\mathcal{A},\mathcal{Z}}(\lambda, L, z)$.

Definition 6. Let KM be a key manager protocol in the $(\mathcal{F}^{\mathsf{tlog}}, \mathcal{F}^{\mathsf{PKI}})$-hybrid model. Then KM securely realizes $\mathcal{F}^{\mathsf{KM}}$ in the $(\mathcal{F}^{\mathsf{tlog}}, \mathcal{F}^{\mathsf{PKI}})$-hybrid model if for all PPT adversaries $\mathcal{A}$, there exists a PPT simulator Sim such that for every PPT environment $\mathcal{Z}$ subject to constraints defined in Appendix B.4.1, and every $\lambda \in \mathbb{N}$, $L \in \mathbb{N}$, and $z \in \{0,1\}^{\mathrm{poly}(\lambda)}$, we have:

$$\Big| \Pr[\mathrm{IDEAL}_{\mathcal{F}^{\mathsf{KM}},\mathsf{Sim},\mathcal{Z}}(\lambda, L, z) = 1] - \Pr[\mathrm{REAL}^{\mathcal{F}^{\mathsf{tlog}},\mathcal{F}^{\mathsf{PKI}}}_{\mathsf{KM},\mathcal{A},\mathcal{Z}}(\lambda, L, z) = 1] \Big| \leq \mathrm{negl}(\lambda)$$

Theorem 7. *As per Definition 6, the key manager protocol* KM *defined in Appendix A.3 securely realizes* $\mathcal{F}^{\mathsf{KM}}$ *in the* $(\mathcal{F}^{\mathsf{tlog}}, \mathcal{F}^{\mathsf{PKI}})$*-hybrid model.*

B.4.3. Notation for the proof (Appendix B.4.4) and simulator (Appendix B.4.5). For a committee $\mathcal{C}_{\mathsf{curr}}$, let $\mathcal{C}^C_{\mathsf{curr}}$ denote the corrupted $\mathcal{C}_{\mathsf{curr}}$-members, and similarly, let $\mathcal{C}^H_{\mathsf{curr}}$ denote the honest $\mathcal{C}_{\mathsf{curr}}$-members. We denote the shares of $\mathcal{C}_{\mathsf{curr}}$-members by $\mathsf{shares}_{\mathsf{curr}}$, and the shares chosen by $\mathcal{M}_i \in \mathcal{C}_{\mathsf{curr}}$ for $\mathcal{C}_{\mathsf{next}}$-members by $\mathsf{shares}_{i\to\mathsf{next}}$. The collection of $O(n^2)$ shares chosen by $\mathcal{C}_{\mathsf{curr}}$ members for $\mathcal{C}_{\mathsf{next}}$-members is denoted by $\mathsf{shares}_{\mathsf{curr}\to\mathsf{next}}$. Similarly, we denote the coefficients of the sharing polynomials chosen by $\mathcal{C}_{\mathsf{curr}}$-members with $\mathsf{coeffs}_{\mathsf{curr}}$. We add a superscript $\{C, H\}$ to the committee labels if we want to denote just the corrupted or honest members on that committee, respectively.

B.4.4. Proof of Theorem 7.

Proof. Now, we argue that with the simulator description above, the ideal and real experiments are indistinguishable:

- H_0 **(The real experiment)**
- H_1 **(Simulate honest parties)**: Sim simulates the honest clients, the server if it is honest, and $\mathcal{F}^{\mathsf{PKI}}$ and $\mathcal{F}^{\mathsf{tlog}}$ functionalities. Since it controls $\mathcal{F}^{\mathsf{PKI}}$, it can register the simulated honest clients with a signature key that it samples. Since it controls honest committee members, for any consecutive epochs prev and curr, Sim knows $\mathsf{shares}_{\mathsf{prev}^H\to\mathsf{curr}^C}$, i.e., random shares chosen by honest $\mathcal{C}_{\mathsf{prev}}$-members for corrupted $\mathcal{C}_{\mathsf{curr}}$-members.

- H_2 **(Pseudorandomness of VRF [76])**: in case the server is honest, Sim aborts if it receives messages from $< t$ honest committee members on any committee. This happens with negligible probability due to the pseudorandomness of VRF outputs of honest parties and parameter selection (Appendix A.2) which guarantees that $\geq t$ honest clients will always be selected with high probability.
- H_3 **(Key-binding property of authenticated dictionary [140]**: Sim aborts (happens with negligible probability) if $\mathcal{A}$ is able to include two or more sets of VRF key-pairs in the client list for the same PKI-registered $(\mathsf{uid}, \mathsf{pk}^{\mathsf{sig}})$ pair.
- H_4 (**Lemma 8**): Sim aborts if the adversary posts a valid committee that has $\geq t$ corrupted clients. This happens with negligible probability from Lemma 8.
- H_5 **(Zero-knowledge property of NIZK [119]**: Sim replaces the following proofs with simulated proofs:
  - PKE well-formedness proof π^{enc} such that the simulated proof does not contain any information about the encryption secret key of honest clients.
  - pVSS proof π^{pv} and aVSS proof π^{ab}, such that the simulated proofs do not contain any information about the shares chosen by an honest committee member for the next committee.
  - Discrete-log equality proof π^{dleq} such that the simulated proof does not contain any information about the secret share of an honest committee member.
- H_6 (**Simulation-extractability of SE-NIZK [85]**): For any consecutive epochs prev and curr, Sim extracts the witness ($\mathsf{coeffs}_{\mathsf{prev}^C}$) underlying the dealings generated by corrupted $\mathcal{C}_{\mathsf{prev}}$-members if an honest $\mathcal{C}_{\mathsf{curr}}$-member does not abort during Reconstruct. Given $\mathsf{shares}_{\mathsf{prev}^H \to \mathsf{curr}^C}$, $\mathsf{coeffs}_{\mathsf{prev}^C}$, and $\mathsf{scmts}_{\mathsf{curr}^C}$ posted by $\mathcal{S}$, Sim interpolates $\mathsf{shares}_{\mathsf{curr}^C}$ which are consistent with $\mathsf{scmts}_{\mathsf{curr}^C}$. Note that the interpolated shares are the same across all honest $\mathcal{C}_{\mathsf{curr}}$-members that do not abort since they are checked against the same $\mathsf{scmts}_{\mathsf{curr}^C}$ posted to the transparency log. Here, we require a straight-line extractor for the (batched) Schnorr Sigma protocol. In our implementation, we make the Sigma protocol non-interactive using the Fiat-Shamir transform [66], and we can obtain a white-box straight line extractor for this NIZK in the Random Oracle Model (ROM) plus the Algebraic Group Model (AGM) through a direct extension of ideas from Fuchsbauer et al. [70]. Assuming the environment does not interact with the AGM oracle for this NIZK, which is the case for our secret recovery protocol, Kerber et al. [99] showed that white-box extraction can be composed. Alternatively, for general composition, one can replace Fiat-Shamir with the Fischlin transform [67].
- H_7 **(Soundness of NIZK [85]**: Sim aborts (happens with negligible probability) if any of the following events happen:
  - An honest committee member accepts a committee even though one of the member's encryption keys was ill-formed.
  - If $\mathcal{S}$ is honest, the server accepts a pVSS proof π^{pv} even though the shares received by honest parties do not match the corresponding coefficient commitment.
  - An honest $\mathcal{C}_2$-member accepts the DL-EQ proof π^{dleq} even though $\mathsf{dlog}_g(\mathsf{ccmt}_{1,i,0}) \neq \mathsf{dlog}_{g'}(\mathsf{scmt}_{1,i})$ for some $\mathcal{M}_i \in \mathcal{C}_{2^C}$.
- H_8 (**EUF-CMA property of signature [80]**): Sim aborts if an honest committee member $\mathcal{M}_i \in \mathcal{C}_{\mathsf{curr}}$ accepts a dealing from an honest committee member $\mathcal{M}_k \in \mathcal{C}_{\mathsf{prev}}$ that $\mathcal{M}_k$ did not generate for epoch curr and $\mathcal{M}_i$.
- H_9 (**mr-IND-CPA property of encryption scheme [96]**): in this hybrid, for any consecutive epochs prev, curr, next and each honest member $\mathcal{M}_i \in \mathcal{C}_{\mathsf{curr}}$, we (i) set the shares $\mathsf{shares}_{i \to \mathsf{next}^H}$ for honest $\mathcal{C}_{\mathsf{next}}$-members to all 0s, and (ii) do not abort when the decrypted share does not match the commitment for honest $\mathcal{C}_{\mathsf{prev}}$-members, and (iii) still output the same share as the previous hybrid to ensure the output consistency. Since PKI setup ensures that the encryption keys of corrupted clients are well-formed, this change is indistinguishable from the previous hybrid due to the mr-IND-CPA property of the encryption scheme.
- H_{10} (**XDH assumption in $\mathbb{G}_1$ [21]**): Sim samples a random $\mathsf{msk} \in \mathbb{F}_q$ and sets $\mathsf{pk} \leftarrow g^{\mathsf{msk}}$. Given msk and shares_{1^C} (note that $|\mathsf{shares}_{1^C}| < t$), Sim interpolates shares'_{1^H} such that when t shares out of $\mathsf{shares}_{1^C} \cup \mathsf{shares}'_{1^H}$ are combined, the result is msk. For each honest client $\mathcal{M}_i \in \mathcal{C}_1^H$ that does not abort, $\mathcal{M}_i$ re-shares $\mathsf{shares}'_{1^H,i}$ instead of $\mathsf{shares}_{1^H_i}$ to $\mathcal{C}_2$. Accordingly, now $\mathcal{M}_i$ provides a simulated π^{dleq} proof on a false statement, which shows that $\mathsf{dlog}_g(g^{\mathsf{shares}'_{1^H,i}}) = \mathsf{dlog}_{g'}(g'^{\mathsf{shares}_{1^H,i}})$ even though they are independent of each other. The adversary will have access to false DL-EQ proofs now, but it is okay because the DL-EQ proof we use are simulation-sound [64].
- H_{11} (**The ideal experiment**): in this hybrid, we introduce the ideal functionality $\mathcal{F}^{\mathsf{KM}}$. Honest parties only talk to $\mathcal{F}^{\mathsf{KM}}$ and the Sim talks to $\mathcal{F}^{\mathsf{KM}}$ on behalf of the corrupted parties. Sim simulates the honest parties for $\mathcal{A}$ and it interacts with $\mathcal{A}$ and $\mathcal{F}^{\mathsf{KM}}$ as detailed in the previous section. Note that this is the same as the ideal experiment. Now, we argue that this hybrid is indistinguishable from the previous hybrid.

  It is easy to see that the simulated honest parties' behavior is identical to hybrid H_{10} from the adversary's perspective. We first argue that the simulator needs access to the same information in both hybrids:
  - In both hybrids, for any honest committee member $\mathcal{M}_i \in \mathcal{C}_{\mathsf{curr}}$, simulated $\mathcal{M}_i$ does not need to know its share

$\mathsf{shares}_{\mathsf{curr},i}$. All information sent by $\mathcal{M}_i$ can be simulated using $\mathsf{sk}_i^{\mathsf{enc}}$ (chosen by Sim), $< t$ shares of corrupted committee members for any epoch curr (i.e., $\mathsf{shares}_{\mathsf{curr}^C}$), and the expected public key pk and the recovery responses resp.

– The simulator does not need secret information to simulate the honest clients, the server, parties fetching pk, and the ideal functionalities.

Now, we argue that the output distribution of honest parties is also identical:

– The simulator can change the public key pk in H_{10} to the one output by $\mathcal{F}^{\mathsf{KM}}$. Since $\mathcal{F}^{\mathsf{KM}}$ generates pk the same way as the Sim does in hybrid H_{10}, this change is indistinguishable. So, the public key output by honest server in H_{10} and the ideal experiment are the same.

– Similarly, Sim can change the responses of honest clients in H_{10} such that they match the recovery response resp output by $\mathcal{F}^{\mathsf{KM}}$ to ensure that the response output by honest server in H_{10} and the ideal experiment are the same.

– Whenever an honest party tries to fetch pk in H_{10}, $\mathcal{F}^{\mathsf{KM}}$ informs simulator and it does the same in the ideal experiment with a simulated party. If this party fails to retrieve pk in H_{10}, Sim ensures the same in the ideal experiment by denying that request on behalf of $\mathcal{A}$.

– In the ideal experiment, if the server is honest, it eventually gets the recovery responses. In H_{10}, the honest server also always gets $\geq t$ valid responses because there are $\geq t$ honest committee members in each committee and their responses are delivered before they time out due to the network assumption we make (see Appendix B.4.1).

– If during any epoch curr, a corrupted server $\mathcal{S}$ deviates from the protocol such that enough honest $\mathcal{C}_{\mathsf{curr}}$-members time out or abort, the server in H_{10} will not get $\geq t$ valid handovers. Consequently, the honest $\mathcal{C}_{\mathsf{next}}$-members will also abort because they verify that there are $\geq t$ valid handovers. This process will go on for all subsequent committees. As such, when Sim detects this condition, it sends $\langle \mathsf{Terminate} \rangle$ to $\mathcal{F}^{\mathsf{KM}}$ because no recoveries can be made this point onwards.

– Finally, Sim informs $\mathcal{F}^{\mathsf{KM}}$ to send outputs when the simulated parties get them in the ideal experiment. So, in the previous hybrid and the ideal experiment, honest parties get their outputs at the same time.

That concludes the proof and we describe the final simulator in Appendix B.4.5. □

Lemma 8. *If an adversary $\mathcal{A}$ that controls the server and at most $\kappa\%$ of clients can output a committee that has $\geq t$ corrupted clients with non-negligible probability, then we can construct an adversary $\mathcal{B}$ that wins the VRF unbiasability game ([76], Definition 8) with non-negligible probability.*

Proof. Without loss of generality, we will only focus on the fairness of committee selection during a single epoch e. We can then use the hybrid argument to extend it to polynomially many epochs. Let $\mathcal{A}_e$ denote the adversary that outputs a committee in epoch e with $\geq t$ corrupted clients with non-negligible probability $\epsilon(\lambda)$. Then we construct an adversary $\mathcal{B}$ that wins the VRF unbiasability game with non-negligible probability. $\mathcal{B}$ simulates the random oracle H used by $\mathcal{A}_e$ to build the Merkle tree, the random oracle H_{seed}, and the ideal functionalities $\mathcal{F}^{\mathsf{tlog}}$ and $\mathcal{F}^{\mathsf{PKI}}$. It works as follows:

- Let Q be the upper bound on the polynomially-many queries made by $\mathcal{A}_e$ to oracle H. $\mathcal{B}$ guesses the index $i \in [Q]$ that will correspond to the root rt of the Merkle tree. Without loss of generality, we can assume that the final root rt chosen by $\mathcal{A}_e$ will be in the H queries. If $\mathcal{A}_e$ does not query it, we can always construct a wrapper adversary that makes this query when $\mathcal{A}_e$ publishes rt to $\mathcal{F}^{\mathsf{tlog}}$ at the end of committee selection.
- It treats the i-th query to H as a root and infers the entire tree based on the other queries made to H. Since a random oracle is collision-resistant, there will be at most one query that is the pre-image of the root, and we can extend this argument all the way to the leaves. If $\mathcal{B}$ cannot construct the tree all the way to the leaves that has the client keys, $\mathcal{B}$ aborts. Else, $\mathcal{B}$ gets to extract c VRF public keys $\{\mathsf{pk}_j^{\mathsf{vrf}}\}_{j\in[c]}$ for the identifiers corresponding to the c corrupted clients in the system. $\mathcal{B}$ knows these identifiers because it simulates $\mathcal{F}^{\mathsf{PKI}}$.
- $\mathcal{B}$ sends these VRF keys to the VRF game challenger along with a PPT computable monotone predicate p that is defined as follows: $p(y_1, \ldots, y_c) = \mathbf{1}\{\sum_{j\in[c]} \mathbf{1}\{y_j \neq \perp \ \wedge \ y_j \leq \frac{n}{N}\} \geq t\}$, i.e., whether the number of keys on the committee exceed the threshold t. $\mathcal{B}$ receives a seed x from the outside challenger, and it programs the output of H_{seed} at input (rt, e) to x.
- Finally, $\mathcal{A}_e$ publishes the committee $\mathcal{C}_e$ on $\mathcal{F}^{\mathsf{tlog}}$. Let rt' be the root published. If the roots don't match, $\mathcal{B}$ aborts. If the honest parties abort while verifying this committee or it has less than t corrupted members, $\mathcal{B}$ aborts. Else, $\mathcal{B}$ takes the VRF outputs and proofs for corrupted parties in $\mathcal{C}_e^C$, puts $\perp$ in place of the output and proof for corrupted parties not on the committee, and forwards them to the outside challenger. This concludes the reduction.

To argue that $\mathcal{B}$ has a non-negligible advantage, observe that when the unbiasability experiment is instantiated with $b = 1$ and p is evaluated on uniformly random values, we have already proved for our parameters that p returns 1 with negligible probability (Appendix A.2). When $b = 0$, note that if $\mathcal{B}$ does not abort, then p returns 1. Thus, we just need to argue that $\mathcal{B}$

does not abort with non-negligible probability. This argument is simple: with probability $1/Q$ which is polynomial in security parameter, we guess the right root. The adversary $\mathcal{A}_e$'s view is identical in the real protocol and in this reduction, and it outputs a committee with $\geq t$ corrupted clients with non-negligible probability $\epsilon(\lambda)$. Thus, with non-negligible probability $\epsilon(\lambda)/Q$, $\mathcal{B}$ does not abort and it wins the VRF unbiasability game. □

B.4.5. Simulator Description. To describe the simulator Sim, we only mention the deltas from the actual protocol. If the server $\mathcal{S}$ is honest, it always follows the actual protocol. Thus, we only focus on how the honest clients which are simulated by Sim behave differently.

$\{\mathcal{C}_e, \perp\} \leftarrow \mathsf{Sim.CommitteeSelection}(e)$:

Same as $\mathsf{ECPSS.CommitteeSelection}(e)$ except:

- **(Client) Setup**: π^{enc} is replaced with a simulated proof.
- **(Client) Get Committee**: Sim aborts if the simulated client does not abort and the committee has $\geq t$ corrupted members.
- If $\mathcal{S}$ is honest, Sim also aborts if it does not receive messages from $\geq t$ honest committee members for any committee.

$\mathsf{shares}_{i\to\mathsf{next}^C} \leftarrow \mathsf{Sim.Share}(\mathsf{scmt}_{\mathsf{curr},i}, \mathcal{C}_{\mathsf{next}})$:

Same as $\mathsf{ECPSS.Share}(\mathsf{shares}_{\mathsf{curr},i}, \mathcal{C}_{\mathsf{next}})$ except Sim does not know $\mathsf{shares}_{\mathsf{curr},i}$ (it only knows commitment to $\mathsf{shares}_{\mathsf{curr},i}$, i.e., $\mathsf{scmt}_{\mathsf{curr},i} = g^{\mathsf{shares}_{\mathsf{curr},i}}$), and as such, it makes the following changes to saVSS.Deal on behalf of $\mathcal{M}_i \in \mathcal{C}_{\mathsf{curr}}$:

- Sim chooses shares $\mathsf{shares}_{i\to\mathsf{next}^C}$ (w.l.o.g., $|\mathsf{shares}_{i\to\mathsf{next}^C}| = t-1$) for corrupted $\mathcal{C}_{\mathsf{next}}$-members on behalf of simulated $\mathcal{M}_i$ at random. From these shares, it gets the corresponding share-commitments $\mathsf{scmts}_{i\to\mathsf{next}^C}$. Given $\mathsf{scmts}_{i\to\mathsf{next}^C}$ and $\mathsf{scmt}_{\mathsf{curr},i}$, Sim interpolates the commitment $\mathsf{ccmt}_{\mathsf{curr},i}$ to coefficients of the sharing polynomial for $\mathcal{M}_i \in \mathcal{C}_{\mathsf{curr}}$ (note that $\mathsf{ccmt}_{\mathsf{curr},i,0} = \mathsf{scmt}_{\mathsf{curr},i}$).
- It sets the shares $\mathsf{shares}_{i\to\mathsf{next}^H}$ of honest $\mathcal{C}_{\mathsf{next}}$-members to all 0.
- Sim generates a simulated π^{ab} proof using the corresponding zero-knowledge simulator w.r.t. cmt. Note that it does not need to know the polynomial coefficients since it's a simulated proof.
- It generates a simulated π^{pv} proof using the corresponding zero-knowledge simulator for the fake statement where cmt does not match the encrypted shares.
- Finally, Sim returns $\mathsf{shares}_{i\to\mathsf{next}^C}$.

$\{\mathsf{shares}_{\mathsf{curr}^C}, \perp\} \leftarrow \mathsf{Sim.Reconstruct}(\mathsf{shares}_{\mathsf{prev}^H\to\mathsf{curr}^C}, \mathcal{C}_{\mathsf{prev}})$:

Same as $\mathsf{ECPSS.Reconstruct}(\mathcal{C}_{\mathsf{prev}})$ except:

- Simulated committee member $\mathcal{M}_i \in \mathcal{C}_{\mathsf{curr}}$ does not abort even if the decrypted share does not match the commitment for dealings from honest committee members (given other checks pass).
- If the simulated honest committee member $\mathcal{M}_i$ does not abort, Sim uses the extractor for π^{ab} to extract the sharing-polynomial coefficients $\mathsf{coeffs}_{\mathsf{prev}^C}$ from the corrupted $\mathcal{C}$-members dealings $\mathcal{M}_i$ receives.
- Given $\mathsf{shares}_{\mathsf{prev}^H\to\mathsf{curr}^C}, \mathsf{coeffs}_{\mathsf{prev}^C}$, and $\mathsf{scmts}_{\mathsf{curr}^C}$ posted by $\mathcal{S}$, Sim interpolates $\mathsf{shares}_{\mathsf{curr}^C}$ which are consistent with $\mathsf{scmts}_{\mathsf{curr}^C}$. It returns $\mathsf{shares}_{\mathsf{curr}^C}$ if $\mathcal{M}_i$ did not abort, else it returns $\perp$.

$(\mathsf{shares}_{\mathsf{curr}^C}, \mathsf{shares}_{\mathsf{curr}^H\to\mathsf{next}^C})$
$\leftarrow \mathsf{Sim.Handover}(\mathsf{pk}, \mathsf{shares}_{\mathsf{prev}^H\to\mathsf{curr}^C}, \mathcal{C}_{\mathsf{prev}}, \mathcal{C}_{\mathsf{curr}}, \mathcal{C}_{\mathsf{next}})$:

Same as $\mathsf{ECPSS.Handover}(\mathcal{C}_{\mathsf{prev}}, \mathcal{C}_{\mathsf{curr}}, \mathcal{C}_{\mathsf{next}})$ except:

- Without loss of generality, we assume that $\mathcal{C}_{\mathsf{curr}}$ has $t-1$ corrupted members. If at least one simulated honest member $\mathcal{M}_i \in \mathcal{C}^H_{\mathsf{curr}}$ does not abort during ECPSS.Reconstruct, Sim gets $\mathsf{shares}_{\mathsf{curr}^C} \leftarrow \mathsf{Sim.Reconstruct}(\mathsf{shares}_{\mathsf{prev}^H\to\mathsf{curr}^C}, \mathcal{C}_{\mathsf{prev}})$. Else, the simulator sends $\langle\mathsf{Terminate}\rangle$ to $\mathcal{F}^{\mathsf{KM}}$ on behalf of corrupted $\mathcal{S}$.
- Based on $\mathsf{shares}_{\mathsf{curr}^C}$ and pk, Sim interpolates the expected share-commitments $\mathsf{scmts}_{\mathsf{curr}}$ w.r.t. generator g. For each honest simulated member $\mathcal{M}_i \in \mathcal{C}^H_{\mathsf{curr}}$ that has not aborted yet, Sim calls $\mathsf{shares}_{i\to\mathsf{next}^C} \leftarrow \mathsf{Sim.Share}(\mathsf{scmts}_{\mathsf{curr},i}, \mathcal{C}_{\mathsf{next}})$.
- Sim defines $\mathsf{shares}_{\mathsf{curr}^H\to\mathsf{next}^C} = \{\mathsf{shares}_{i\to\mathsf{next}^C}\}_{\mathcal{M}_i\in\mathcal{C}^H_{\mathsf{curr}}}$, and returns $\mathsf{shares}_{\mathsf{curr}^C}$ for higher-level protocol and $\mathsf{shares}_{\mathsf{curr}^H\to\mathsf{next}^C}$ for the next handover.

$\mathsf{KM.KeyGen}()$

- If $\mathcal{S}$ is corrupted, Sim sends $\langle\mathsf{KeyGen}\rangle$ to $\mathcal{F}^{\mathsf{KM}}$ on behalf of the corrupted server. Irrespective of whether the server is corrupted, Sim receives $\langle\mathsf{KeyGen}, \mathsf{pk}\rangle$ from $\mathcal{F}^{\mathsf{KM}}$.
- **Epoch 0**: Sim follows the protocol, invoking ECPSS.Share from simulated honest committee members in $\mathcal{C}_0$, and stores $\mathsf{shares}_{0^H\to1^C}$.
- **Epoch 1**: Epoch 1 proceeds the same way as the protocol, except honest parties follow $(\mathsf{shares}_{1^C}, \mathsf{shares}_{1^H\to2^C}) \leftarrow \mathsf{Sim.Handover}(\mathsf{pk}, \mathsf{shares}_{0^H\to1^C}, \mathcal{C}_0, \mathcal{C}_1, \mathcal{C}_2)$, where each simulated client uses generator g' when invoking Sim.Reconstruct and generator g when invoking Sim.Share. Each simulated client $\mathcal{M}_i \in \mathcal{C}^H_1$ that does not abort also replaces π^{dleq} with

a simulated proof for a fake statement, which shows that $\mathsf{dlog}_g(\mathsf{ccmt}_{1,i,0}) = \mathsf{dlog}_{g'}(\mathsf{scmt}_{1,i})$ even though the former depends on pk while the latter is independent of it.

- If $\mathcal{S}$ is honest, when the simulated $\mathcal{S}$ interpolates pk, Sim responds to $\mathcal{F}^{\mathsf{KM}}$ on behalf of $\mathcal{A}$ to forward pk to honest server. Else, Sim sends this response when the corrupted server posts pk to the transparency log at the end of epoch 1.

KM.PublicKey()

- When $\mathcal{F}^{\mathsf{KM}}$ receives a $\langle \mathsf{PublicKey} \rangle$ request from an honest party $\mathcal{P}$, $\mathcal{F}^{\mathsf{KM}}$ informs Sim on behalf of $\mathcal{A}$. Sim then tries to run the get public key protocol while simulating $\mathcal{P}$. If it is able to retrieve pk, Sim sends $\top$ to $\mathcal{F}^{\mathsf{KM}}$ on behalf of $\mathcal{A}$; else it sends $\perp$. (only happens if $\mathcal{S}$ is corrupted).

KM.Recover$(\mathsf{rqst}, \mathsf{st}_{\mathsf{new}}, \pi^{\mathsf{st}})$

The protocol proceeds the same way except:

- If $\mathcal{S}$ is honest, Sim receives $\langle \mathsf{Recover}, \mathsf{rqst}, \mathsf{st}_{\mathsf{new}}, \pi^{\mathsf{st}}, \mathsf{resp} \rangle$ from $\mathcal{F}^{\mathsf{KM}}$ on behalf of $\mathcal{A}$. The simulated server initiates the processing of this request in the coming epoch curr.
- In epoch curr, honest parties follow $(\mathsf{shares}_{\mathsf{curr}^C}, \mathsf{shares}_{\mathsf{curr}^H \to \mathsf{next}^C}) \leftarrow \mathsf{Sim.Handover}(\mathsf{pk}, \mathsf{shares}_{\mathsf{prev}^H \to \mathsf{curr}^C}, \mathcal{C}_{\mathsf{prev}}, \mathcal{C}_{\mathsf{curr}}, \mathcal{C}_{\mathsf{next}})$ instead of $\mathsf{ECPSS.Handover}(\mathcal{C}_{\mathsf{prev}}, \mathcal{C}_{\mathsf{curr}}, \mathcal{C}_{\mathsf{next}})$.
- If $\mathcal{S}$ is corrupted and a simulated committee member $\mathcal{M}_i \in \mathcal{C}^H_{\mathsf{curr}}$ receives $(\mathsf{rqst}, \mathsf{st}_{\mathsf{prev}}, \mathsf{st}_{\mathsf{curr}}, \pi^{\mathsf{st}})$ and does not abort for epoch curr (Sim sends $\langle \mathsf{Terminate} \rangle$ if all abort), Sim sends $\langle \mathsf{Recover}, \mathsf{rqst}, \mathsf{st}_{\mathsf{new}}, \pi^{\mathsf{st}} \rangle$ on behalf of corrupted $\mathcal{S}$ to $\mathcal{F}^{\mathsf{KM}}$. In response, it receives $\langle \mathsf{Recover}, \mathsf{rqst}, \mathsf{st}_{\mathsf{new}}, \pi^{\mathsf{st}}, \mathsf{resp} \rangle$ from $\mathcal{F}^{\mathsf{KM}}$. Note that irrespective of server corruption, Sim now has resp corresponding to rqst.
- From Sim.Handover, Sim knows $\mathsf{shares}_{\mathsf{curr}^C}$. Given resp and these shares, it can interpolate the expected response resp_i for each simulated $\mathcal{M}_i \in \mathcal{C}^H_{\mathsf{curr}}$, and this is what the simulated $\mathcal{M}_i$ sends back to the server.
- If $\mathcal{S}$ is honest, when the simulated $\mathcal{S}$ interpolates resp, Sim informs $\mathcal{F}^{\mathsf{KM}}$ on behalf of $\mathcal{A}$ to forward resp to honest server. Else, Sim informs $\mathcal{F}^{\mathsf{KM}}$ to move forward when the corrupted $\mathcal{S}$ posts a new rate-limiting state to the transparency log.